COGNIZANT DIGITAL NURTURE

SPRING BOOT & REST SERVICES ASSESSMENT

Week 3 Handson Solutions Report

CANDIDATE INFORMATION

Candidate Name: Ishan Parmar

Student ID: 12239022

Weekly Topics: Spring REST, Jakarta Beans Validation, Custom Security & JJWT Auth

GitHub Repository Link: github.com/ishanparmar2105-stack/digital-nurture-project

Project Scope and Objectives:

This assessment demonstrates hands-on experience in building enterprise Spring Boot REST API endpoints, implementing metadata loading from XML configurations, executing server-side request body and field validations using Jakarta annotations, developing a robust global handler to return standardized REST error formats, and implementing advanced modern Spring Security using basic credentials and JJWT JSON Web Tokens with Bearer Token Authorization Filters.

Section 1: Introduction and Core Architecture

The Maven project 'spring-learn' integrates several key Spring Boot layers:
1. Representation Layer: RestControllers serving endpoints for Hello, Country, Department, and Employee entities.
2. Configuration Layer: XML definitions loaded at application startup to populate singleton, prototype, and collection bean structures.
3. Security Layer: In-memory accounts with ADMIN/USER roles. Authentication is handled by issuing a custom HS256 HMAC-SHA JWT token which is validated by a downstream Bearer token filter on each secure resource request.
4. Validation Layer: Custom GlobalExceptionHandler intercepting validations and bad JSON input structures, guaranteeing consistent API feedback.

Section 2: Spring XML Configuration Files

File: date-format.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

</beans>
```

File: country.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Hands on 4 default country bean -->
    <bean id="country" class="com.cognizant.springlearn.model.Country" scope="prototype">
        <property name="code" value="IN" />
        <property name="name" value="India" />
    </bean>

    <!-- Four country beans for Hands on 6 -->
    <bean id="in" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="IN" />
        <property name="name" value="India" />
    </bean>

    <bean id="us" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="US" />
        <property name="name" value="United States" />
    </bean>

    <bean id="de" class="com.cognizant.springlearn.model.Country">
```

```
        <property name="code" value="DE" />
        <property name="name" value="Germany" />
    </bean>

    <bean id="jp" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="JP" />
        <property name="name" value="Japan" />
    </bean>

    <bean id="countryList" class="java.util.ArrayList">
        <constructor-arg>
            <list>
                <ref bean="in" />
                <ref bean="us" />
                <ref bean="de" />
                <ref bean="jp" />
            </list>
        </constructor-arg>
    </bean>

</beans>
```

File: employee.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Departments -->
    <bean id="deptEng" class="com.cognizant.springlearn.model.Department">
        <property name="id" value="1" />
        <property name="name" value="Engineering" />
    </bean>

    <bean id="deptHR" class="com.cognizant.springlearn.model.Department">
        <property name="id" value="2" />
        <property name="name" value="Human Resources" />
    </bean>

    <bean id="deptFin" class="com.cognizant.springlearn.model.Department">
        <property name="id" value="3" />
        <property name="name" value="Finance" />
    </bean>

    <bean id="departmentList" class="java.util.ArrayList">
        <constructor-arg>
            <list>
                <ref bean="deptEng" />
                <ref bean="deptHR" />
                <ref bean="deptFin" />
            </list>
        </constructor-arg>
    </bean>

    <!-- Skills -->
    <bean id="skillJava" class="com.cognizant.springlearn.model.Skill">
        <property name="id" value="101" />
        <property name="name" value="Java" />
```

```
    </bean>

    <bean id="skillSpring" class="com.cognizant.springlearn.model.Skill">
        <property name="id" value="102" />
        <property name="name" value="Spring Boot" />
    </bean>

    <bean id="skillSQL" class="com.cognizant.springlearn.model.Skill">
        <property name="id" value="103" />
        <property name="name" value="SQL" />
    </bean>

    <!-- Employees -->
    <bean id="emp1" class="com.cognizant.springlearn.model.Employee">
        <property name="id" value="1" />
        <property name="name" value="John Doe" />
        <property name="salary" value="75000.0" />
        <property name="permanent" value="true" />
        <property name="dateOfBirth" value="15/08/1995" />
        <property name="department" ref="deptEng" />
        <property name="skills">
            <list>
                <ref bean="skillJava" />
                <ref bean="skillSpring" />
            </list>
        </property>
    </bean>

    <bean id="emp2" class="com.cognizant.springlearn.model.Employee">
        <property name="id" value="2" />
        <property name="name" value="Alice Smith" />
        <property name="salary" value="82000.0" />
        <property name="permanent" value="true" />
        <property name="dateOfBirth" value="23/11/1993" />
        <property name="department" ref="deptEng" />
        <property name="skills">
            <list>
                <ref bean="skillJava" />
                <ref bean="skillSQL" />
            </list>
        </property>
    </bean>

    <bean id="emp3" class="com.cognizant.springlearn.model.Employee">
        <property name="id" value="3" />
        <property name="name" value="Bob Jones" />
        <property name="salary" value="58000.0" />
        <property name="permanent" value="false" />
        <property name="dateOfBirth" value="10/02/1998" />
        <property name="department" ref="deptHR" />
        <property name="skills">
            <list>
                <ref bean="skillSQL" />
            </list>
        </property>
    </bean>

    <bean id="emp4" class="com.cognizant.springlearn.model.Employee">
        <property name="id" value="4" />
        <property name="name" value="Charlie Brown" />
        <property name="salary" value="65000.0" />
```

```
        <property name="permanent" value="true" />
        <property name="dateOfBirth" value="05/05/1996" />
        <property name="department" ref="deptFin" />
        <property name="skills">
            <list>
                <ref bean="skillSpring" />
                <ref bean="skillSQL" />
            </list>
        </property>
    </bean>

    <bean id="employeeList" class="java.util.ArrayList">
        <constructor-arg>
            <list>
                <ref bean="emp1" />
                <ref bean="emp2" />
                <ref bean="emp3" />
                <ref bean="emp4" />
            </list>
        </constructor-arg>
    </bean>

</beans>
```

Section 3: Validation Models & Entities

File: Country.java

```
package com.cognizant.springlearn.model;

import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Country {
    private static final Logger LOGGER = LoggerFactory.getLogger(Country.class);

    @NotNull(message = "Country code should not be null")
    @Size(min = 2, max = 2, message = "Country code should be 2 characters")
    private String code;
    private String name;

    public Country() {
        LOGGER.debug("Inside Country Constructor.");
    }

    public Country(String code, String name) {
        LOGGER.debug("Inside Country Parameterized Constructor.");
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        LOGGER.debug("Inside getCode Getter.");
        return code;
    }

    public void setCode(String code) {
        LOGGER.debug("Inside setCode Setter: {}", code);
        this.code = code;
    }

    public String getName() {
        LOGGER.debug("Inside getName Getter.");
        return name;
    }

    public void setName(String name) {
        LOGGER.debug("Inside setName Setter: {}", name);
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

File: Employee.java

```
package com.cognizant.springlearn.model;
```

```
import com.fasterxml.jackson.annotation.JsonFormat;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import java.util.List;

public class Employee {
    @NotNull(message = "Employee ID cannot be null")
    private Integer id;

    @NotNull(message = "Employee name cannot be null")
    @NotBlank(message = "Employee name cannot be blank")
    @Size(min = 1, max = 30, message = "Employee name length must be between 1 and 30 characters")
    private String name;

    @NotNull(message = "Employee salary cannot be null")
    @Min(value = 0, message = "Employee salary must be zero or above")
    private Double salary;

    @NotNull(message = "Employee permanent status cannot be null")
    private Boolean permanent;

    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "dd/MM/yyyy")
    private String dateOfBirth;

    @Valid
    @NotNull(message = "Employee department cannot be null")
    private Department department;

    @Valid
    private List<Skill> skills;

    public Employee() {}

    public Employee(Integer id, String name, Double salary, Boolean permanent, String dateOfBirth,
Department department, List<Skill> skills) {
        this.id = id;
        this.name = name;
        this.salary = salary;
        this.permanent = permanent;
        this.dateOfBirth = dateOfBirth;
        this.department = department;
        this.skills = skills;
    }

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
```

```
    }

    public Double getSalary() {
        return salary;
    }

    public void setSalary(Double salary) {
        this.salary = salary;
    }

    public Boolean getPermanent() {
        return permanent;
    }

    public void setPermanent(Boolean permanent) {
        this.permanent = permanent;
    }

    public String getDateOfBirth() {
        return dateOfBirth;
    }

    public void setDateOfBirth(String dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    public Department getDepartment() {
        return department;
    }

    public void setDepartment(Department department) {
        this.department = department;
    }

    public List<Skill> getSkills() {
        return skills;
    }

    public void setSkills(List<Skill> skills) {
        this.skills = skills;
    }

    @Override
    public String toString() {
        return "Employee [id=" + id + ", name=" + name + ", salary=" + salary + ", permanent=" +
permanent
                + ", dateOfBirth=" + dateOfBirth + ", department=" + department + ", skills=" + skills +
"]";
    }
}
```

Section 4: Spring Security & JJWT Authentication Configuration

File: SecurityConfig.java

```
package com.cognizant.springlearn.security;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {
    private static final Logger LOGGER = LoggerFactory.getLogger(SecurityConfig.class);

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {
        LOGGER.info("START: Configuring In-Memory Users");
        UserDetails admin = User.withUsername("admin")
                .password(passwordEncoder().encode("pwd"))
                .roles("ADMIN")
                .build();

        UserDetails user = User.withUsername("user")
                .password(passwordEncoder().encode("pwd"))
                .roles("USER")
                .build();
        LOGGER.info("END: Configured admin and user accounts");
        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        LOGGER.info("START: Configuring PasswordEncoder");
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, AuthenticationConfiguration
authConfiguration) throws Exception {
        LOGGER.info("START: Configuring SecurityFilterChain");

        AuthenticationManager authManager = authConfiguration.getAuthenticationManager();

        http.csrf(csrf -> csrf.disable())
            .httpBasic(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
                .requestMatchers("/countries").hasRole("USER")
                .requestMatchers("/error").permitAll()
                .anyRequest().authenticated()
            )
            .addFilter(new JwtAuthorizationFilter(authManager));

        LOGGER.info("END: Configured SecurityFilterChain successfully");
        return http.build();
    }
}
```

File: JwtAuthorizationFilter.java

```
package com.cognizant.springlearn.security;

import com.cognizant.springlearn.controller.AuthenticationController;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jws;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.security.Key;
import java.util.ArrayList;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.www.BasicAuthenticationFilter;

public class JwtAuthorizationFilter extends BasicAuthenticationFilter {
    private static final Logger LOGGER = LoggerFactory.getLogger(JwtAuthorizationFilter.class);
    private static final Key SIGNING_KEY =
Keys.hmacShaKeyFor(AuthenticationController.SECRET_KEY_STRING.getBytes(StandardCharsets.UTF_8));

    public JwtAuthorizationFilter(AuthenticationManager authenticationManager) {
        super(authenticationManager);
        LOGGER.info("START: JwtAuthorizationFilter Constructor");
        LOGGER.debug("AuthenticationManager: {}", authenticationManager);
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws IOException, ServletException {
        LOGGER.info("START: doFilterInternal()");
        String header = req.getHeader("Authorization");
        LOGGER.debug("Authorization Header: {}", header);

        if (header == null || !header.startsWith("Bearer ")) {
            LOGGER.info("No Bearer token found, proceeding filter chain");
            chain.doFilter(req, res);
            return;
        }
```

```
        UsernamePasswordAuthenticationToken authentication = getAuthentication(req);
        if (authentication != null) {
            SecurityContextHolder.getContext().setAuthentication(authentication);
            LOGGER.info("Successfully set Authentication into Security Context for user: {}",
authentication.getPrincipal());
        } else {
            LOGGER.warn("Invalid JWT token. Failed to authenticate.");
        }

        chain.doFilter(req, res);
        LOGGER.info("END: doFilterInternal()");
    }

    private UsernamePasswordAuthenticationToken getAuthentication(HttpServletRequest request) {
        String token = request.getHeader("Authorization");
        if (token != null) {
            try {
                String cleanToken = token.replace("Bearer ", "").trim();
                Jws<Claims> jws = Jwts.parserBuilder()
                        .setSigningKey(SIGNING_KEY)
                        .build()
                        .parseClaimsJws(cleanToken);

                String user = jws.getBody().getSubject();
                LOGGER.debug("Parsed JWT User: {}", user);

                if (user != null) {
                    java.util.List<org.springframework.security.core.authority.SimpleGrantedAuthority>
authorities = new java.util.ArrayList<>();
                    if ("admin".equals(user)) {
                        authorities.add(new
org.springframework.security.core.authority.SimpleGrantedAuthority("ROLE_ADMIN"));
                    } else if ("user".equals(user)) {
                        authorities.add(new
org.springframework.security.core.authority.SimpleGrantedAuthority("ROLE_USER"));
                    }
                    return new UsernamePasswordAuthenticationToken(user, null, authorities);
                }
            } catch (JwtException ex) {
                LOGGER.error("JwtException caught while parsing token: {}", ex.getMessage());
                return null;
            }
        }
        return null;
    }
}
```

File: AuthenticationController.java

```
package com.cognizant.springlearn.controller;

import io.jsonwebtoken.JwtBuilder;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import java.nio.charset.StandardCharsets;
import java.security.Key;
import java.util.Base64;
import java.util.Date;
```

```
import java.util.HashMap;
import java.util.Map;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AuthenticationController {
    private static final Logger LOGGER = LoggerFactory.getLogger(AuthenticationController.class);
    public static final String SECRET_KEY_STRING =
"secretkey-must-be-at-least-256-bits-long-for-hmac-sha-signing-algorithms-123456789";
    private static final Key SIGNING_KEY =
Keys.hmacShaKeyFor(SECRET_KEY_STRING.getBytes(StandardCharsets.UTF_8));

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(@RequestHeader("Authorization") String authHeader) {
        LOGGER.info("START: authenticate()");
        LOGGER.debug("Authorization Header: {}", authHeader);

        String user = getUser(authHeader);
        String token = generateJwt(user);

        Map<String, String> response = new HashMap<>();
        response.put("token", token);

        LOGGER.info("END: authenticate() returned token successfully");
        return response;
    }

    private String getUser(String authHeader) {
        LOGGER.info("START: getUser()");
        // authHeader: "Basic YWRtaW46cHdk"
        String base64Credentials = authHeader.substring("Basic ".length()).trim();
        byte[] decodedBytes = Base64.getDecoder().decode(base64Credentials);
        String credentials = new String(decodedBytes, StandardCharsets.UTF_8);

        // credentials: "username:password"
        String[] values = credentials.split(":", 2);
        String user = values[0];

        LOGGER.info("END: getUser() identified user: {}", user);
        return user;
    }

    private String generateJwt(String user) {
        LOGGER.info("START: generateJwt(user: {})", user);

        JwtBuilder builder = Jwts.builder();
        builder.setSubject(user);
        builder.setIssuedAt(new Date());
        // Set expiry as 20 minutes from now (1200000 ms)
        builder.setExpiration(new Date(System.currentTimeMillis() + 1200000));
        builder.signWith(SIGNING_KEY, SignatureAlgorithm.HS256);

        String token = builder.compact();
        LOGGER.info("END: generateJwt() generated token");
        return token;
    }
}
```

Section 5: REST Controllers & Global Exception Handlers

File: CountryController.java

```
package com.cognizant.springlearn.controller;

import com.cognizant.springlearn.exception.CountryNotFoundException;
import com.cognizant.springlearn.model.Country;
import com.cognizant.springlearn.service.CountryService;
import jakarta.validation.Valid;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {
    private static final Logger LOGGER = LoggerFactory.getLogger(CountryController.class);

    @Autowired
    private CountryService countryService;

    @GetMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START: getCountryIndia()");
        ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
        Country india = context.getBean("in", Country.class);
        LOGGER.info("END: getCountryIndia() returned: {}", india);
        return india;
    }

    @GetMapping("/countries")
    public List<Country> getAllCountries() {
        LOGGER.info("START: getAllCountries()");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.info("END: getAllCountries() returned {} countries", countries.size());
        return countries;
    }

    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) throws CountryNotFoundException {
        LOGGER.info("START: getCountry(code: {})", code);
        Country country = countryService.getCountry(code);
        LOGGER.info("END: getCountry() returned: {}", country);
        return country;
    }

    @PostMapping("/countries")
    public Country addCountry(@RequestBody @Valid Country country) {
        LOGGER.info("START: addCountry(country: {})", country);
        // Validations are automatically performed via @Valid in Spring Boot 3
        LOGGER.info("END: addCountry(country: {}) successfully completed", country);
        return country;
```

```
    }
}
```

File: EmployeeController.java

```
package com.cognizant.springlearn.controller;

import com.cognizant.springlearn.exception.EmployeeNotFoundException;
import com.cognizant.springlearn.model.Employee;
import com.cognizant.springlearn.service.EmployeeService;
import jakarta.validation.Valid;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class EmployeeController {
    private static final Logger LOGGER = LoggerFactory.getLogger(EmployeeController.class);

    @Autowired
    private EmployeeService employeeService;

    @GetMapping("/employees")
    public List<Employee> getAllEmployees() {
        LOGGER.info("START: getAllEmployees() controller method");
        List<Employee> list = employeeService.getAllEmployees();
        LOGGER.info("END: getAllEmployees() returned {} records", list.size());
        return list;
    }

    @PutMapping("/employees")
    public void updateEmployee(@RequestBody @Valid Employee employee) throws EmployeeNotFoundException {
        LOGGER.info("START: updateEmployee() controller method: {}", employee.getId());
        employeeService.updateEmployee(employee);
        LOGGER.info("END: updateEmployee() controller method finished");
    }

    @DeleteMapping("/employees/{id}")
    public void deleteEmployee(@PathVariable int id) throws EmployeeNotFoundException {
        LOGGER.info("START: deleteEmployee() controller method: {}", id);
        employeeService.deleteEmployee(id);
        LOGGER.info("END: deleteEmployee() controller method finished");
    }
}
```

File: GlobalExceptionHandler.java

```
package com.cognizant.springlearn.exception;

import com.fasterxml.jackson.databind.exc.InvalidFormatException;
import java.util.ArrayList;
import java.util.Date;
```

```
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {
    private static final Logger LOGGER = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex, HttpHeaders headers, HttpStatusCode status, WebRequest
request) {
        LOGGER.info("Start: handleMethodArgumentNotValid");

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());

        // Get all validation errors
        List<String> errors = ex.getBindingResult().getFieldErrors().stream()
                .map(x -> x.getDefaultMessage())
                .collect(Collectors.toList());

        body.put("errors", errors);
        LOGGER.info("End: handleMethodArgumentNotValid: errors={}", errors);
        return new ResponseEntity<>(body, headers, status);
    }

    @Override
    protected ResponseEntity<Object> handleHttpMessageNotReadable(
            HttpMessageNotReadableException ex, HttpHeaders headers, HttpStatusCode status, WebRequest
request) {
        LOGGER.info("Start: handleHttpMessageNotReadable");

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", status.value());
        body.put("error", "Bad Request");

        if (ex.getCause() instanceof InvalidFormatException) {
            final Throwable cause = ex.getCause() == null ? ex : ex.getCause();
            for (InvalidFormatException.Reference reference : ((InvalidFormatException) cause).getPath())
{
                body.put("message", "Incorrect format for field '" + reference.getFieldName() + "'");
            }
        }
        LOGGER.info("End: handleHttpMessageNotReadable");
        return new ResponseEntity<>(body, headers, status);
    }
```

```
    @org.springframework.web.bind.annotation.ExceptionHandler(CountryNotFoundException.class)
    public ResponseEntity<Object> handleCountryNotFound(CountryNotFoundException ex) {
        LOGGER.info("START: handleCountryNotFound");
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", 404);
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());
        LOGGER.info("END: handleCountryNotFound");
        return new ResponseEntity<>(body, org.springframework.http.HttpStatus.NOT_FOUND);
    }

    @org.springframework.web.bind.annotation.ExceptionHandler(EmployeeNotFoundException.class)
    public ResponseEntity<Object> handleEmployeeNotFound(EmployeeNotFoundException ex) {
        LOGGER.info("START: handleEmployeeNotFound");
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", new Date());
        body.put("status", 404);
        body.put("error", "Not Found");
        body.put("message", ex.getMessage());
        LOGGER.info("END: handleEmployeeNotFound");
        return new ResponseEntity<>(body, org.springframework.http.HttpStatus.NOT_FOUND);
    }
}
```

Section 6: JUnit MockMVC Testing Suite

File: SpringLearnApplicationTests.java

```
package com.cognizant.springlearn;

import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import com.cognizant.springlearn.controller.CountryController;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.security.test.context.support.WithMockUser;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.ResultActions;

@SpringBootTest
@AutoConfigureMockMvc
public class SpringLearnApplicationTests {

    @Autowired
    private CountryController countryController;

    @Autowired
    private MockMvc mvc;

    @Test
    public void contextLoads() {
        assertNotNull(countryController);
    }

    @Test
    @WithMockUser(username = "user", roles = {"USER"})
    public void testGetCountry() throws Exception {
        ResultActions actions = mvc.perform(get("/country"));
        actions.andExpect(status().isOk());
        actions.andExpect(jsonPath("$.code").exists());
        actions.andExpect(jsonPath("$.code").value("IN"));
        actions.andExpect(jsonPath("$.name").exists());
        actions.andExpect(jsonPath("$.name").value("India"));
    }

    @Test
    @WithMockUser(username = "user", roles = {"USER"})
    public void testGetCountryException() throws Exception {
        ResultActions actions = mvc.perform(get("/countries/az"));
        // Assert status is 404 (Not Found)
        actions.andExpect(status().isNotFound());
    }
}
```

Section 7: Verification Tests Execution Output & Curl Logs

API Curl Requests & Responses Log

```
Starting REST Endpoint Verification Tests using curl...
================================================================================
TEST: 1. Hello Endpoint - Unauthorized Check
================================================================================
COMMAND: curl -i http://localhost:8090/hello
HTTP STATUS: HTTP/1.1 401
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  Set-Cookie: JSESSIONID=040E3D66867FE90B466E2620150A05C7; Path=/; HttpOnly
  WWW-Authenticate: Basic realm="Realm"
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "timestamp": "2026-07-07T18:08:02.185+00:00",
  "status": 401,
  "error": "Unauthorized",
  "message": "Unauthorized",
  "path": "/hello"
}



================================================================================
TEST: 2. Hello Endpoint - Basic Auth Authorized
================================================================================
COMMAND: curl -i -u user:pwd http://localhost:8090/hello
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: text/plain;charset=UTF-8
  Content-Length: 13
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
Hello World!!



================================================================================
TEST: 3. GET /country (India details)
================================================================================
COMMAND: curl -i -u user:pwd http://localhost:8090/country
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
```

```
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "code": "IN",
  "name": "India"
}



================================================================================
TEST: 4. GET /countries (List of countries)
================================================================================
COMMAND: curl -i -u user:pwd http://localhost:8090/countries
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "DE",
    "name": "Germany"
  },
  {
    "code": "JP",
    "name": "Japan"
  }
]



================================================================================
TEST: 5a. GET /countries/in (Case-insensitive check)
================================================================================
COMMAND: curl -i -u user:pwd http://localhost:8090/countries/in
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
```

```
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "code": "IN",
  "name": "India"
}



================================================================================
TEST: 5b. GET /countries/IN (Case-insensitive check)
================================================================================
COMMAND: curl -i -u user:pwd http://localhost:8090/countries/IN
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "code": "IN",
  "name": "India"
}



================================================================================
TEST: 6. GET /countries/az (Exception scenario)
================================================================================
COMMAND: curl -i -u user:pwd http://localhost:8090/countries/az
HTTP STATUS: HTTP/1.1 404
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "timestamp": "2026-07-07T18:08:02.768+00:00",
  "status": 404,
  "error": "Not Found",
  "message": "Country not found"
}



================================================================================
TEST: 7. GET /authenticate (JWT Token generation)
================================================================================
```

```
COMMAND: curl -i -u user:pwd http://localhost:8090/authenticate
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "token":
"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98Mk
PkB5k1w8FAALoVRBcg"
}



Retrieved token: eyJhbGciOiJIUzI1NiJ9...

================================================================================
TEST: 8. GET /countries (using JWT Bearer Token)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg http://localhost:8090/countries
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "DE",
    "name": "Germany"
  },
  {
    "code": "JP",
    "name": "Japan"
  }
]



================================================================================
TEST: 9. POST /countries - Valid payload
```

```
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -H Content-Type: application/json -X POST -d {"code":"ES","name":"Spain"}
http://localhost:8090/countries
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "code": "ES",
  "name": "Spain"
}



================================================================================
TEST: 10. POST /countries - Invalid code validation error
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -H Content-Type: application/json -X POST -d {"code":"S","name":"Spain"}
http://localhost:8090/countries
HTTP STATUS: HTTP/1.1 400
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
  Connection: close
RESPONSE BODY:
{
  "timestamp": "2026-07-07T18:08:02.936+00:00",
  "status": 400,
  "errors": [
    "Country code should be 2 characters"
  ]
}



================================================================================
TEST: 11. GET /departments (Static departments list)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg http://localhost:8090/departments
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
```

```
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[
  {
    "id": 1,
    "name": "Engineering"
  },
  {
    "id": 2,
    "name": "Human Resources"
  },
  {
    "id": 3,
    "name": "Finance"
  }
]



================================================================================
TEST: 12. GET /employees (Static employees list)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg http://localhost:8090/employees
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[
  {
    "id": 1,
    "name": "John Doe",
    "salary": 75000.0,
    "permanent": true,
    "dateOfBirth": "15/08/1995",
    "department": {
      "id": 1,
      "name": "Engineering"
    },
    "skills": [
      {
        "id": 101,
        "name": "Java"
      },
      {
        "id": 102,
        "name": "Spring Boot"
      }
    ]
```

```
  },
  {
    "id": 2,
    "name": "Alice Smith",
    "salary": 82000.0,
    "permanent": true,
    "dateOfBirth": "23/11/1993",
    "department": {
      "id": 1,
      "name": "Engineering"
    },
    "skills": [
      {
        "id": 101,
        "name": "Java"
      },
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  },
  {
    "id": 3,
    "name": "Bob Jones",
    "salary": 58000.0,
    "permanent": false,
    "dateOfBirth": "10/02/1998",
    "department": {
      "id": 2,
      "name": "Human Resources"
    },
    "skills": [
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  },
  {
    "id": 4,
    "name": "Charlie Brown",
    "salary": 65000.0,
    "permanent": true,
    "dateOfBirth": "05/05/1996",
    "department": {
      "id": 3,
      "name": "Finance"
    },
    "skills": [
      {
        "id": 102,
        "name": "Spring Boot"
      },
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  }
]



================================================================================
TEST: 13. PUT /employees - Valid update
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -H Content-Type: application/json -X PUT -d {"id": 1, "name": "John Updated", "salary":
```

```
95000.0, "permanent": true, "dateOfBirth": "15/08/1995", "department": {"id": 1, "name": "Engineering"},
"skills": [{"id": 101, "name": "Java"}, {"id": 102, "name": "Spring Boot"}]} http://localhost:8090/employees
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Length: 0
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[Empty Body]



================================================================================
TEST: 14. GET /employees (Verify PUT changes)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg http://localhost:8090/employees
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[
  {
    "id": 1,
    "name": "John Updated",
    "salary": 95000.0,
    "permanent": true,
    "dateOfBirth": "15/08/1995",
    "department": {
      "id": 1,
      "name": "Engineering"
    },
    "skills": [
      {
        "id": 101,
        "name": "Java"
      },
      {
        "id": 102,
        "name": "Spring Boot"
      }
    ]
  },
  {
    "id": 2,
    "name": "Alice Smith",
    "salary": 82000.0,
    "permanent": true,
    "dateOfBirth": "23/11/1993",
    "department": {
```

```
      "id": 1,
      "name": "Engineering"
    },
    "skills": [
      {
        "id": 101,
        "name": "Java"
      },
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  },
  {
    "id": 3,
    "name": "Bob Jones",
    "salary": 58000.0,
    "permanent": false,
    "dateOfBirth": "10/02/1998",
    "department": {
      "id": 2,
      "name": "Human Resources"
    },
    "skills": [
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  },
  {
    "id": 4,
    "name": "Charlie Brown",
    "salary": 65000.0,
    "permanent": true,
    "dateOfBirth": "05/05/1996",
    "department": {
      "id": 3,
      "name": "Finance"
    },
    "skills": [
      {
        "id": 102,
        "name": "Spring Boot"
      },
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  }
]



================================================================================
TEST: 15. PUT /employees - Invalid salary validation error
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -H Content-Type: application/json -X PUT -d {"id": 1, "name": "John Updated", "salary":
-500.0, "permanent": true, "dateOfBirth": "15/08/1995", "department": {"id": 1, "name": "Engineering"},
"skills": [{"id": 101, "name": "Java"}, {"id": 102, "name": "Spring Boot"}]} http://localhost:8090/employees
HTTP STATUS: HTTP/1.1 400
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
```

```
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
  Connection: close
RESPONSE BODY:
{
  "timestamp": "2026-07-07T18:08:03.011+00:00",
  "status": 400,
  "errors": [
    "Employee salary must be zero or above"
  ]
}



================================================================================
TEST: 16. PUT /employees - Bad format deserialization error
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -H Content-Type: application/json -X PUT -d {"id":"abc","name":"Bad
Format","salary":80000.0,"permanent":true,"dateOfBirth":"15/08/1995"} http://localhost:8090/employees
HTTP STATUS: HTTP/1.1 400
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
  Connection: close
RESPONSE BODY:
{
  "timestamp": "2026-07-07T18:08:03.022+00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Incorrect format for field 'id'"
}



================================================================================
TEST: 17. DELETE /employees/3 (Valid deletion)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -X DELETE http://localhost:8090/employees/3
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Length: 0
```

```
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[Empty Body]



================================================================================
TEST: 18. GET /employees (Verify DELETE changes)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg http://localhost:8090/employees
HTTP STATUS: HTTP/1.1 200
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
[
  {
    "id": 1,
    "name": "John Updated",
    "salary": 95000.0,
    "permanent": true,
    "dateOfBirth": "15/08/1995",
    "department": {
      "id": 1,
      "name": "Engineering"
    },
    "skills": [
      {
        "id": 101,
        "name": "Java"
      },
      {
        "id": 102,
        "name": "Spring Boot"
      }
    ]
  },
  {
    "id": 2,
    "name": "Alice Smith",
    "salary": 82000.0,
    "permanent": true,
    "dateOfBirth": "23/11/1993",
    "department": {
      "id": 1,
      "name": "Engineering"
    },
    "skills": [
      {
        "id": 101,
        "name": "Java"
      },
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  },
```

```
  {
    "id": 4,
    "name": "Charlie Brown",
    "salary": 65000.0,
    "permanent": true,
    "dateOfBirth": "05/05/1996",
    "department": {
      "id": 3,
      "name": "Finance"
    },
    "skills": [
      {
        "id": 102,
        "name": "Spring Boot"
      },
      {
        "id": 103,
        "name": "SQL"
      }
    ]
  }
]



================================================================================
TEST: 19. DELETE /employees/99 (Exception scenario)
================================================================================
COMMAND: curl -i -H Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNzgzNDQ3NjgyLCJleHAiOjE3ODM0NDg4ODJ9.XDq9D2JUiAbf6WyvdaV6x98MkP
kB5k1w8FAALoVRBcg -X DELETE http://localhost:8090/employees/99
HTTP STATUS: HTTP/1.1 404
RESPONSE HEADERS:
  Vary: Origin
  Vary: Access-Control-Request-Method
  Vary: Access-Control-Request-Headers
  X-Content-Type-Options: nosniff
  X-XSS-Protection: 0
  Cache-Control: no-cache, no-store, max-age=0, must-revalidate
  Pragma: no-cache
  Expires: 0
  X-Frame-Options: DENY
  Content-Type: application/json
  Transfer-Encoding: chunked
  Date: Tue, 07 Jul 2026 18:08:02 GMT
RESPONSE BODY:
{
  "timestamp": "2026-07-07T18:08:03.049+00:00",
  "status": 404,
  "error": "Not Found",
  "message": "Employee not found"
}
```